2026–2027 PROFESSIONAL EDITION (VERSION 2.0)

Volume I: The Complete CKA Textbook

Mastering Kubernetes Administration, Architecture, Workloads & Scheduling (80 Chapters)

Cloud Native Engineering Press

Targeting Kubernetes v1.31.x / v1.32.x (Forward Compatible through v1.33+)

Independent Professional Certification Study System

Certified Kubernetes Administrator (CNCF / The Linux Foundation)

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART I: KUBERNETES FUNDAMENTALS

CHAPTERS 1–4: CONTAINERIZATION, OCI ARCHITECTURE & CLUSTER COMPONENTS

CHAPTER 1: INTRODUCTION TO CONTAINERIZATION AND KUBERNETES

1. Learning Objectives

- Understand the evolution from bare metal and virtual machines to containers.
- Identify the operational bottlenecks that Kubernetes was engineered to solve.
- Articulate the core responsibilities of a production container orchestrator.

2. Prerequisites

- Familiarity with Linux command-line basics (`ls` , `ps` , `cat` , `systemctl`).
- Basic comprehension of operating system concepts: kernel, user space, and process execution.

3. Beginner-Friendly Introduction: Why Do Containers and Kubernetes Exist?

In the early days of computing, applications ran directly on bare-metal servers. A server running an operating system like Linux or Windows ran all binaries together. If an application leaked memory or monopolized the CPU, every other process on that host starved. Furthermore, dependency conflicts—such as Application A requiring Python 3.8 and Application B requiring Python 3.12—made server management brittle.

Virtual Machines (VMs) solved isolation by placing a hypervisor (e.g., KVM, VMware) on the hardware and running entire independent guest operating systems. However, VMs carry massive overhead: every VM requires gigabytes of disk storage for its guest kernel, takes minutes to boot, and wastes gigabytes of RAM running duplicate operating system services.

Containers offer an elegant, lightweight alternative: **Process-level virtualization**. Containers do not run a guest kernel; they run as isolated processes directly on the host Linux kernel, sharing its CPU and memory while maintaining strict namespace boundaries.

However, running hundreds of standalone containers across dozens of physical servers creates new operational challenges:

- What happens when a container crashes at 3:00 AM?

- How do containers discover and communicate with each other across different servers?
- How do we roll out new application versions without dropping customer traffic?
- Which server has enough CPU and memory to host a new container?

Kubernetes is the open-source container orchestration system that automates deployment, scaling, healing, and management of containerized workloads.

4. Detailed Conceptual Explanation & Internal Architecture

At the kernel level, a container is not an actual physical object; it is a standard Linux process isolated by two fundamental kernel subsystems:

1. **Linux Namespaces (Isolation):** Restricts *what a process can see*.
    - `pid`: Process IDs (a containerized process believes it is PID 1).
    - `net`: Network interfaces, IP addresses, routing tables, and firewall rules.
    - `mnt`: Filesystem mount points (isolates root `/` via pivot_root).
    - `ipc`: System V IPC and POSIX message queues.
    - `uts`: Hostname and NIS domain name.
    - `user`: Maps container root (UID 0) to an unprivileged user on the host.
2. **Control Groups / Cgroups (Resource Metering):** Restricts *how much a process can use*.
    - Limits maximum CPU shares, CFS bandwidth quotas, memory caps, and disk I/O throughput.

Kubernetes abstracts these low-level Linux primitives into resilient, high-level API objects.

CHAPTER 2: CONTAINER ARCHITECTURE AND OCI STANDARDS

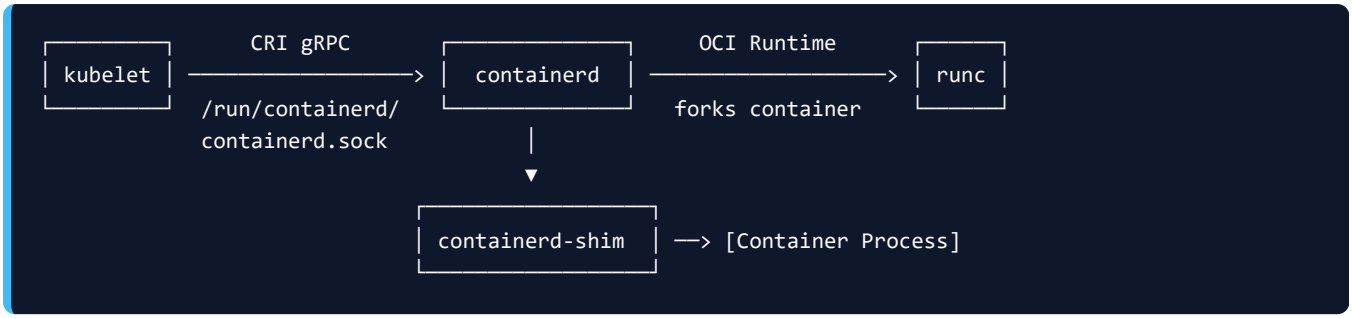
1. The Open Container Initiative (OCI)

To avoid vendor lock-in, the industry established the Open Container Initiative (OCI), which governs two core standards:

1. **OCI Image Specification:** Defines the layout and format of container images (tarballs containing root filesystems and JSON manifest descriptors).
2. **OCI Runtime Specification:** Defines how to unpack and execute an OCI image as a set of Linux namespaces and cgroups (implemented by low-level runtimes like `runc` and `crun`).

2. The Container Runtime Interface (CRI)

In early Kubernetes releases, the `kubelet` directly integrated with Docker via a hardcoded internal bridge called `dockershim`. In Kubernetes v1.24+, `dockershim` was completely removed. Modern Kubernetes communicates with container runtimes strictly through the **Container Runtime Interface (CRI)**, a gRPC API.



3. Container Runtime Troubleshooting Workflow

When a node reports container runtime issues, verify containerd with:

```
# Verify systemd service status
systemctl status containerd

# Query active pod sandboxes via CRI CLI
crictl pods

# Check containerd configuration socket
ls -la /run/containerd/containerd.sock
```

CHAPTER 3: KUBERNETES ARCHITECTURE & HIGH-LEVEL DESIGN

1. Control Plane vs. Worker Nodes

A Kubernetes cluster consists of two distinct operational tiers:

1. **The Control Plane:** The cluster's nervous system. It makes global decisions (scheduling, detecting node failures, maintaining desired replica counts) and stores the definitive state.
2. **Worker Nodes:** The compute workhorses. They host the application containers, provide network routing, and execute health checks.

```
graph TB
    subgraph Control_Plane["Control Plane"]
        API["kube-apiserver (Port 6443)"]
        ETCD[("etcd (Port 2379)")]
        Sched["kube-scheduler"]
        KCM["kube-controller-manager"]
        API <--> ETCD
        Sched --> API
        KCM --> API
    end
    subgraph Worker_Node["Worker Node"]
        Kubelet["kubelet"]
        KProxy["kube-proxy"]
        CRI["containerd"]
        Pods["Workload Pods"]
        Kubelet --> API
        KProxy --> API
        Kubelet --> CRI
        CRI --> Pods
    end
```

CHAPTER 4: CONTROL PLANE AND WORKER NODE COMPONENTS DEEP DIVE

1. kube-apiserver

The central REST API gateway. Every administrative command from `kubectl`, every scheduling binding from `kube-scheduler`, and every heartbeat from `kubelet` connects to `kube-apiserver`.

- **State Statelessness:** The API server stores no state locally; all data is persisted in `etcd`.
- **Security Gate:** Enforces Authentication, Authorization (RBAC), and Admission Control.

2. etcd

A distributed, consistent key-value store implementing the Raft consensus algorithm. It is the single source of truth for the entire cluster.

- **Ports:** Port 2379 (Client requests from apiserver); Port 2380 (Peer communication among etcd members).
- **Quorum Rule:** Requires $\lfloor N/2 \rfloor + 1$ members to commit writes.

3. kube-scheduler

Watches for newly created Pods that have no `nodeName` assigned. It evaluates filtering predicates (hard constraints) and scoring priorities (soft preferences) to select the optimal worker node.

4. kube-controller-manager (KCM)

A single binary compiling multiple control loops (Replication Controller, Node Controller, Endpoints Controller, ServiceAccount Controller). Each loop continuously drives actual cluster state toward the desired state.

5. kubelet (Worker Node Agent)

Runs as a native `systemd` daemon on every node (not inside a container).

- Interprets `PodSpec` objects and calls CRI to create/destroy containers.
- Mounts storage volumes and configures network namespaces.
- Continuously executes liveness, readiness, and startup probes.
- Posts node health heartbeats (Leases) back to the API server.

6. kube-proxy (Network Director)

Runs on every node (typically as a DaemonSet). Watches Services and EndpointSlices, configuring host `iptables` or `IPVS` rules to load-balance traffic directed at Service virtual IPs across healthy backend Pods.

PRACTICE EXERCISES & KNOWLEDGE CHECK (CHAPTERS 1–4)

Part A: Knowledge Check Questions

1. Why was `dockershim` removed from Kubernetes, and what interface replaced it?
2. Which control plane component communicates directly with `etcd`?
3. What is the minimum number of nodes required to survive 2 simultaneous node failures in an etcd cluster?
4. Explain the difference between Linux namespaces and cgroups.

Part B: Solutions

1. `dockershim` was removed to decouple Kubernetes from proprietary Docker code; it was replaced by the standardized Container Runtime Interface (CRI).
2. Only `kube-apiserver` communicates with `etcd`.
3. An etcd cluster requires 5 nodes to tolerate 2 node failures ($5/2 + 1 = 3$ quorum required).
4. Namespaces isolate visibility (what a process sees); cgroups meter and throttle resource consumption (how much CPU/RAM it can use).

Part C: Revision Checklist

- [ ] Memorized the 6 primary cluster components.
- [ ] Understand why kubelet runs as a systemd service while other components run as static pods.
- [ ] Able to run `crictl pods` and `systemctl status containerd` on a broken node.

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART I: KUBERNETES FUNDAMENTALS

CHAPTERS 5–8: API PRIMITIVES, DECLARATIVE YAML, KUBECTL & LINUX NETWORKING

CHAPTER 5: KUBERNETES API SERVER AND API OBJECTS

1. Learning Objectives

- Understand how API resources are grouped, versioned, and addressed.
- Trace the HTTP REST request pipeline from client to etcd.
- Differentiate between core API groups (`/api/v1`) and named groups (`/apis/<group>/<version>`).

2. API Groups and Versioning Architecture

The Kubernetes API is organized hierarchically into:

1. **Core Group (Legacy):** Located at `/api/v1` . Houses original primitives: `Pod` , `Service` , `Namespace` , `ConfigMap` , `Secret` , `Node` , `PersistentVolume` .
2. **Named Groups:** Located at `/apis/<group>/<version>` . Examples:
    - `apps/v1` : Deployments, ReplicaSets, StatefulSets, DaemonSets.
    - `batch/v1` : Jobs, CronJobs.
    - `networking.k8s.io/v1` : Ingress, NetworkPolicy.
    - `storage.k8s.io/v1` : StorageClass, VolumeAttachment.
    - `rbac.authorization.k8s.io/v1` : Role, ClusterRole, RoleBinding.
    - `gateway.networking.k8s.io/v1` : Gateway, HTTPRoute.

3. API Version Lifecycles

- `v1alpha1` : Experimental; subject to breaking changes or removal without notice.
- `v1beta1` : Well-tested; enabled by default; deprecated prior to GA.
- `v1` (GA / Stable): Fully supported across minor versions with strict backward compatibility.

CHAPTER 6: DECLARATIVE CONFIGURATION AND YAML

1. The Anatomy of a Kubernetes Manifest

Every Kubernetes resource requires four top-level fields:

```
apiVersion: apps/v1        # 1. API Group and Version
kind: Deployment           # 2. Resource Kind
metadata:                  # 3. Object Identification
  name: payment-processor
  namespace: finance
  labels:
    app: payment
spec:                      # 4. Desired State Specification
  replicas: 3
  selector:
    matchLabels:
      app: payment
  template:
    metadata:
      labels:
        app: payment
    spec:
      containers:
      - name: processor
        image: payment-app:v2.1
        ports:
        - containerPort: 8080
```

2. Imperative vs. Declarative Management

- **Imperative Commands:** `kubectl create deployment ...`, `kubectl scale ...`. Excellent for speed during the exam to generate initial manifests.
- **Declarative Management:** `kubectl apply -f manifest.yaml`. Computes a 3-way diff between the local file, the live etcd record, and the `kubectl.kubernetes.io/last-applied-configuration` annotation.

CHAPTER 7: KUBECTL ADMINISTRATION AND EXAM PRODUCTIVITY

1. Essential Imperative Generators

Never write YAML from scratch in the CKA exam. Use these standard generators:

```
# Generate Pod YAML
kubectl run nginx-pod --image=nginx:alpine --port=80 --dry-run=client -o yaml > pod.yaml

# Generate Deployment YAML
kubectl create deployment web-app --image=nginx:alpine --replicas=3 --dry-run=client -o yaml > deploy.yam

# Generate Service YAML (ClusterIP)
kubectl expose deployment web-app --port=80 --target-port=80 --dry-run=client -o yaml > svc.yaml

# Generate Job YAML
kubectl create job batch-calc --image=busybox --dry-run=client -o yaml -- sh -c "expr 5 + 5" > job.yaml

# Generate Namespace
kubectl create namespace staging --dry-run=client -o yaml > ns.yaml
```

2. Advanced JSONPath Extraction

Extract nested values directly from API server outputs:

```
# Get all Pod IPs in default namespace
kubectl get pods -o jsonpath='{.items[*].status.podIP}'

# Get all Node names and their Internal IPs
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.addresses[?(@.type=="Inter

# Find container images used in all deployments
kubectl get deployments -A -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.template.spec.conta
```

CHAPTER 8: LINUX AND NETWORKING FOUNDATIONS FOR KUBERNETES

1. The Fundamental Kubernetes Network Model

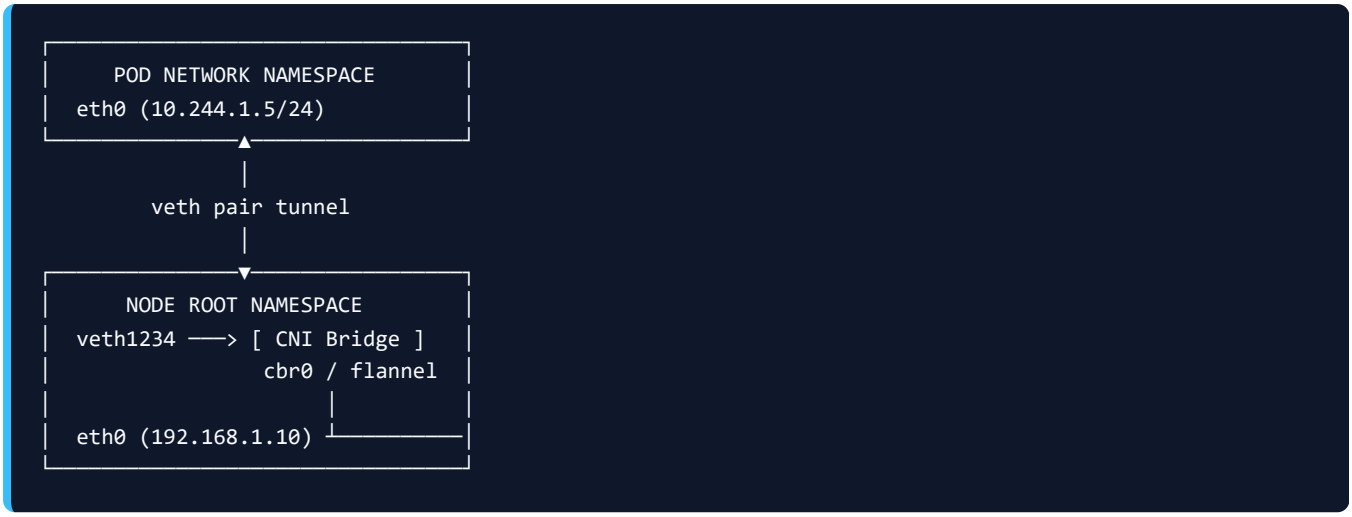
Kubernetes imposes four non-negotiable networking requirements:

1. Every Pod gets its own unique IP address.
2. Pods on any node can communicate with all pods on any other node without NAT.
3. Agents on a node (kubelet) can communicate with all pods on that same node.
4. Pods in the host network of a node can communicate with all pods on all nodes without NAT.

2. Virtual Ethernet Pairs (veth) and Network Namespaces

When a Pod is scheduled:

1. The container runtime creates an isolated Linux network namespace (`netns`).
2. A virtual ethernet pair (`veth` pair) is created in the kernel. One end (`eth0`) is moved into the Pod's network namespace, and the peer end (`vethXXXX`) is plugged into the node's local bridge (e.g., `cbr0`).
3. An IP address is allocated from the node's `podCIDR` allocation by the CNI IPAM plugin.



3. Linux Kernel Diagnostics Runbook

```
# Check network namespace links
ip netns list

# Inspect Linux bridges and interfaces
ip link show
ip addr show

# Check host routing table
ip route show

# Verify socket listening ports
ss -tulpen
```

PRACTICE EXERCISES & CKA CHALLENGE (CHAPTERS 5–8)

Challenge: Multi-Container Pod with Declarative YAML

Task: Create a Pod named `log-processor` in namespace `observability`:

1. Main container `app` running `busybox:1.36` writing timestamped logs every 5 seconds to `/var/log/app.log`.
2. Sidecar container `logger` running `busybox:1.36` reading and streaming `/var/log/app.log` to standard output.
3. Both containers share a volume named `shared-logs` of type `emptyDir`.

Solution Manifest:

```
apiVersion: v1
kind: Namespace
metadata:
  name: observability
---
apiVersion: v1
kind: Pod
metadata:
  name: log-processor
  namespace: observability
spec:
  volumes:
  - name: shared-logs
    emptyDir: {}
  containers:
  - name: app
    image: busybox:1.36
    command: ["/bin/sh", "-c"]
    args:
    - >
      while true; do
        echo "$(date) - Application event processing" >> /var/log/app.log;
        sleep 5;
      done
    volumeMounts:
    - name: shared-logs
      mountPath: /var/log
  - name: logger
    image: busybox:1.36
    command: ["/bin/sh", "-c"]
    args: ["tail -n+1 -f /var/log/app.log"]
    volumeMounts:
    - name: shared-logs
      mountPath: /var/log
```

Verification:

```
kubectl apply -f solution.yaml
kubectl logs -n observability log-processor -c logger
```

Expected output: Continuous stream of timestamped log entries originating from the `app` container.

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART II: CLUSTER ARCHITECTURE, INSTALLATION & LIFECYCLE

CHAPTERS 9–14, 17–19: KUBEADM, CONTAINER RUNTIME, UPGRADES & NODE MAINTENANCE

CHAPTER 9: KUBERNETES CLUSTER DESIGN & HARDWARE SIZING

1. Learning Objectives

- Calculate control plane sizing based on nodes, pods, and API churn.
- Evaluate stacked etcd vs. external etcd architectures.
- Prepare Linux operating system prerequisites for Kubernetes clusters.

2. Linux Operating System Prerequisites

Every Kubernetes node (control plane or worker) requires kernel module configuration and sysctl networking parameters:

```
# 1. Disable Linux Swap (Mandatory for Kubelet memory management)
sudo swapoff -a
sudo sed -i '/swap/d' /etc/fstab

# 2. Forwarding IPv4 and letting iptables see bridged traffic
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter

# 3. Sysctl parameters required by CNI
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

sudo sysctl --system
```

CHAPTER 10 & 11: CONTAINER RUNTIME & KUBEADM INSTALLATION

1. Configuring containerd with SystemdCgroup

Kubernetes v1.31/v1.32 strictly requires that `containerd` and `kubelet` use the `systemd` cgroup driver to prevent resource starvation under memory pressure:

```
# Generate default containerd config
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml

# Set SystemdCgroup = true
sudo sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g' /etc/containerd/config.toml

# Restart containerd service
sudo systemctl restart containerd
sudo systemctl enable containerd
```

2. Installing Kubernetes Binaries via Official Community Repository

```
# 1. Add Kubernetes package repository key
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyri

# 2. Add repository source
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.3

# 3. Install packages and hold version
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

CHAPTER 12–14: CONTROL PLANE INITIALIZATION & WORKER JOINS

1. Initializing the First Control Plane Node

```
sudo kubeadm init \
  --apiserver-advertise-address=192.168.1.10 \
  --pod-network-cidr=10.244.0.0/16 \
  --cri-socket=unix:///run/containerd/containerd.sock \
  --upload-certs
```

2. Post-Initialization Configuration

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

3. Installing CNI Network Plugin (Calico)

The cluster remains in `NotReady` status until a CNI network plugin is deployed:

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml
```

4. Joining Worker Nodes

On each worker node, run the command generated by `kubeadm init`:

```
sudo kubeadm join 192.168.1.10:6443 --token abcdef.0123456789abcdef \
  --discovery-token-ca-cert-hash sha256:7c8b4e2f9d1a3b5c7e9f8a...
```

CHAPTER 17 & 18: CLUSTER UPGRADES & SAFE NODE MAINTENANCE

1. Zero-Downtime Node Maintenance Algorithm

When patching an OS kernel or upgrading node components, follow the three-step lifecycle:

1. **Cordon:** Prevents new workloads from being scheduled on the node.

```
kubectl cordon worker-01
```

2. **Drain:** Safely evicts existing Pods using the Eviction API, respecting `PodDisruptionBudgets`.

```
kubectl drain worker-01 --ignore-daemonsets --delete-emptydir-data --force
```

3. **Perform Maintenance & Uncordon:**

```
# Upgrade node packages, reboot if necessary, then restore scheduling
kubectl uncordon worker-01
```

2. Kubeadm Minor Version Upgrade Sequence

Always upgrade the Control Plane before Worker Nodes. Never skip minor versions (e.g., upgrade v1.30 -> v1.31 -> v1.32 sequentially):

```
# --- ON CONTROL PLANE NODE ---
# 1. Upgrade kubeadm
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.31.2-1.1
sudo apt-mark hold kubeadm

# 2. Check upgrade plan
sudo kubeadm upgrade plan

# 3. Apply cluster upgrade
sudo kubeadm upgrade apply v1.31.2 -y

# 4. Drain node & upgrade kubelet/kubectl
kubectl drain controlplane --ignore-daemonsets
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo apt-mark hold kubelet kubectl

sudo systemctl daemon-reload
sudo systemctl restart kubelet
kubectl uncordon controlplane

# --- ON WORKER NODES ---
kubectl drain worker-01 --ignore-daemonsets --delete-emptydir-data
ssh worker-01
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.31.2-1.1
sudo apt-mark hold kubeadm
sudo kubeadm upgrade node
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo apt-mark hold kubelet kubectl
sudo systemctl daemon-reload
sudo systemctl restart kubelet
exit
kubectl uncordon worker-01
```

CHAPTER 19: CERTIFICATE MANAGEMENT & ROTATION

Control plane certificates issued by `kubeadm` expire after 365 days.

```
# 1. Check certificate validity
sudo kubeadm certs check-expiration

# 2. Renew all certificates
sudo kubeadm certs renew all

# 3. Reload static pods
sudo mv /etc/kubernetes/manifests/*.yaml /tmp/
sleep 5
sudo mv /tmp/*.yaml /etc/kubernetes/manifests/

# 4. Update admin kubeconfig
sudo cp /etc/kubernetes/admin.conf ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
```

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART II: CLUSTER ARCHITECTURE, INSTALLATION & LIFECYCLE

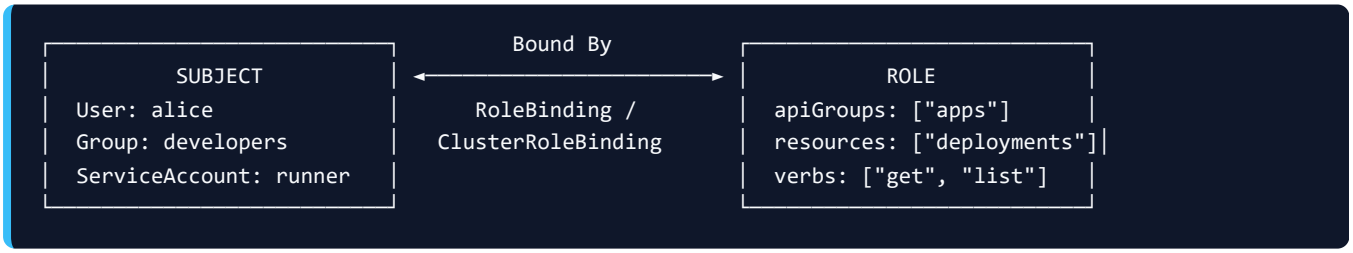
CHAPTERS 15, 16, 20–23: RBAC, SERVICEACCOUNTS, ETCD DISASTER RECOVERY, HELM & KUSTOMIZE

CHAPTER 15: ROLE-BASED ACCESS CONTROL (RBAC)

1. Learning Objectives

- Understand the RBAC evaluation model: Subjects, API Groups, Resources, and Verbs.
- Contrast namespace-scoped `Roles` with cluster-wide `ClusterRoles`.
- Bind permissions using `RoleBindings` and `ClusterRoleBindings`.

2. The RBAC Data Model



3. Production RBAC Manifest Example

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: engineering
  name: deployment-operator
rules:
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: engineering
  name: bind-deployment-operator
subjects:
- kind: User
  name: bob
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: ci-bot
  namespace: engineering
roleRef:
  kind: Role
  name: deployment-operator
  apiGroup: rbac.authorization.k8s.io
```

CHAPTER 16: SERVICEACCOUNTS AND POD AUTHENTICATION

1. ServiceAccount Token Projection

Modern Kubernetes (v1.24+) does not automatically create static bearer token secrets for ServiceAccounts. Instead, tokens are projected dynamically into pods as projected volumes using the TokenRequest API:

- Tokens have an expiration time (default 1 hour).
- Tokens are automatically rotated by kubelet without restarting the container.
- Bound directly to the Pod's lifecycle.

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-reader-sa
  namespace: default
---
apiVersion: v1
kind: Pod
metadata:
  name: app-reader-pod
spec:
  serviceAccountName: app-reader-sa
  containers:
  - name: runner
    image: curlimages/curl:8.10.1
    command: ["sleep", "3600"]
```

CHAPTER 20: HIGHLY AVAILABLE KUBERNETES CLUSTERS

1. Stacked vs. External etcd Topologies

1. **Stacked etcd Topology:** etcd runs as static pods on the same nodes as the control plane components. Simpler to manage via `kubeadm init --upload-certs`. Requires at least 3 control plane nodes.
2. **External etcd Topology:** etcd runs on dedicated external servers. Decouples control plane compute from etcd storage I/O.

2. Control Plane Load Balancer

A high-availability control plane requires a TCP load balancer (e.g., HAProxy + Keepalived) in front of the API servers:

- Virtual IP: `192.168.1.100:6443`
- Backends: `cp-node-1:6443`, `cp-node-2:6443`, `cp-node-3:6443`
- Pass `--control-plane-endpoint="192.168.1.100:6443"` to `kubeadm init`.

CHAPTER 21: ETCD ARCHITECTURE, BACKUP AND DISASTER RECOVERY

1. The Definitive 7-Step etcd Disaster Recovery Runbook

This is the single most heavily tested administrative procedure on the CKA examination.

```
# STEP 1: Identify etcd Static Pod Configuration & PKI Paths
sudo grep -E "cert-file|key-file|trusted-ca-file|data-dir" /etc/kubernetes/manifests/etcd.yaml

# STEP 2: Save Snapshot using etcdctl v3
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /opt/snapshot-pre-disaster.db

# STEP 3: Verify Snapshot Integrity
ETCDCTL_API=3 etcdctl --write-out=table snapshot status /opt/snapshot-pre-disaster.db

# STEP 4: Stop kube-apiserver by moving manifest temporarily
sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/

# STEP 5: Restore Snapshot to a BRAND NEW Target Data Directory
ETCDCTL_API=3 etcdctl snapshot restore /opt/snapshot-pre-disaster.db \
  --data-dir=/var/lib/etcd-from-backup \
  --initial-cluster="master=https://127.0.0.1:2380" \
  --initial-cluster-token="etcd-cluster-token" \
  --initial-advertise-peer-urls="https://127.0.0.1:2380" \
  --name="master"

# STEP 6: Update etcd Static Pod Manifest hostPath Volume
sudo sed -i 's|path: /var/lib/etcd|path: /var/lib/etcd-from-backup|g' /etc/kubernetes/manifests/etcd.yaml

# STEP 7: Restore kube-apiserver & Verify Cluster Health
sudo mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/
sleep 20
kubectl get nodes
```

CHAPTER 22: HELM AND KUSTOMIZE PACKAGE MANAGEMENT

1. Helm Package Manager Fundamentals

Helm is the official package manager for Kubernetes.

```
# Add and update repositories
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

# Install chart with custom values
helm install my-ingress ingress-nginx/ingress-nginx \
  --namespace ingress-system --create-namespace \
  --set controller.replicaCount=2 \
  -f custom-values.yaml

# Upgrade release
helm upgrade my-ingress ingress-nginx/ingress-nginx --set controller.replicaCount=3

# Inspect release history and rollback
helm history my-ingress -n ingress-system
helm rollback my-ingress 1 -n ingress-system
```

2. Kustomize Overlay Architecture

Kustomize enables template-free customization of Kubernetes manifests via `kustomization.yaml` files.

```
app/
├── base/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
└── overlays/
    ├── dev/
    │   └── kustomization.yaml
    └── prod/
        ├── kustomization.yaml
        └── replica_patch.yaml
```

`base/kustomization.yaml`:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
```

`overlays/prod/kustomization.yaml`:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- ../../base
namePrefix: prod-
patchesStrategicMerge:
- replica_patch.yaml
```

Applying Kustomize:

```
kubectl apply -k overlays/prod/
```

CHAPTER 23: CRDS, OPERATORS AND EXTENSION INTERFACES

1. Custom Resource Definitions (CRDs)

CRDs extend the Kubernetes API with user-defined schemas using OpenAPI v3 validation:

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: backups.storage.acme.io
spec:
  group: storage.acme.io
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              schedule:
                type: string
  scope: Namespaced
  names:
    plural: backups
    singular: backup
    kind: Backup
```

2. The Operator Pattern

An Operator pairs a Custom Resource with a custom Controller. The controller watches events on the CRD and automates state management (e.g., PostgreSQL backup, automated failover, schema migrations).

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART III: WORKLOADS AND SCHEDULING

CHAPTERS 24–29, 31: PODS, DEPLOYMENTS, CONTROLLERS, CONFIG & HEALTH PROBES

CHAPTER 24 & 25: PODS, POD LIFECYCLE & MULTI-CONTAINER PATTERNS

1. Pod Phase Transitions

A Pod's lifecycle is tracked via `status.phase`:

- `Pending`: Pod accepted by the API server, but one or more containers are not yet created (often awaiting scheduling or image download).
- `Running`: Pod bound to a node, all containers created, and at least one container is running or starting.
- `Succeeded`: All containers terminated successfully (exit code 0); cannot restart.
- `Failed`: All containers terminated, with at least one container terminating in failure (non-zero exit code).
- `Unknown`: State cannot be obtained (typically due to network loss between kubelet and API server).

2. Multi-Container Pod Design Patterns

1. **Sidecar Pattern:** Enhances or extends the main container (e.g., streaming logs from an `emptyDir` volume to centralized Elasticsearch or Prometheus metrics exporter).
2. **Ambassador Pattern:** Proxies network traffic from the local pod to the outside world (e.g., connecting to `localhost:6379` while the ambassador routes to a sharded Redis cluster).
3. **Adapter Pattern:** Standardizes output across disparate application formats (e.g., transforming legacy application metric formats into Prometheus exposition format).
4. **Init Containers:** Sequential initialization containers that run to complete exit 0 before application containers start.

CHAPTER 26: DEPLOYMENTS AND REPLICASETS

1. Deployment Strategies: Recreate vs. RollingUpdate

- **Recreate Strategy:** Terminates all existing pods before creating new pods (`spec.strategy.type: Recreate`). Results in downtime but prevents two application versions from executing concurrently.
- **RollingUpdate Strategy:** Gradually replaces old pods with new pods. Governed by:
  - `maxSurge` : Max pods created above desired count (default 25%).
  - `maxUnavailable` : Max pods unavailable during update (default 25%).

2. Declarative Deployment Manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  labels:
    app: orders
spec:
  replicas: 4
  revisionHistoryLimit: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: orders
  template:
    metadata:
      labels:
        app: orders
    spec:
      containers:
      - name: orders
        image: orders-api:v1.2.0
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: 250m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 256Mi
```

CHAPTER 27 & 28: DAEMONSETS, STATEFULSETS, JOBS & CRONJOBS

1. StatefulSet Production Considerations

- Headless Service (`clusterIP: None`) provides stable DNS records: `$(pod-name).$(service-name).$(namespace).svc.cluster.local` .
- `volumeClaimTemplates` dynamically provisions a unique, dedicated PersistentVolumeClaim for each ordinal replica.
- Deleting a StatefulSet *never* deletes its associated PersistentVolumeClaims to protect stateful data from accidental loss.

2. CronJob Concurrency Policies

Governs how new executions behave if the previous scheduled Job has not completed:

- `Allow` (Default): Concurrent jobs run simultaneously.
- `Forbid` : Skips the new execution if the previous run is still active.
- `Replace` : Cancels the currently running job and starts a new one.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hourly-aggregation
spec:
  schedule: "0 * * * *"
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 120
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: aggregator
            image: analytics:1.0
```

CHAPTER 29: CONFIGMAPS AND SECRETS

1. Base64 Secret Management

Kubernetes Secrets store sensitive configuration encoded in Base64:

```
# Encode secret value
echo -n "P@ssw0rd123" | base64

# Decode secret value
echo -n "UEBzc3cwcmQxMjM=" | base64 --decode
```

2. Projecting ConfigMaps and Secrets as Volume Mounts

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
spec:
  volumes:
  - name: config-vol
    configMap:
      name: app-config
  - name: secret-vol
    secret:
      secretName: db-secret
  containers:
  - name: app
    image: nginx:alpine
    volumeMounts:
    - name: config-vol
      mountPath: /etc/config
      readOnly: true
    - name: secret-vol
      mountPath: /etc/secrets
      readOnly: true
```

CHAPTER 31: LIVENESS, READINESS AND STARTUP PROBES

```
spec:
  containers:
  - name: web
    image: web-service:2.0
    # 1. Startup Probe: Allows slow apps up to 300s to boot without premature kills
    startupProbe:
      httpGet:
        path: /healthz
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
    # 2. Liveness Probe: Detects runtime deadlocks; restarts container on failure
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20
      failureThreshold: 3
    # 3. Readiness Probe: Controls traffic routing; isolates pod from endpoints on failure
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      periodSeconds: 5
      failureThreshold: 2
```

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART III: WORKLOADS AND SCHEDULING

CHAPTERS 30, 32–38: SCHEDULING, AFFINITY, TAINTS, AUTOSCALING & QUOTAS

CHAPTER 30: RESOURCE REQUESTS, LIMITS & KERNEL CFS QUOTAS

1. Compute Units in Kubernetes

- **CPU:** Specified in millicores ($1000\text{m} = 1\text{ vCPU / core}$). It is a compressible resource: exceeding limits results in CPU cycle throttling via Linux CFS, not termination.
- **Memory:** Specified in bytes with binary SI suffixes (`128Mi` , `1Gi`). It is an incompressible resource: exceeding limits results in an instant kernel `OOMKilled` (Exit Code 137).

2. Quality of Service (QoS) Classes

Kubernetes assigns each Pod a QoS class based on its request/limit declarations:

1. **Guaranteed (Highest Priority):** Every container specifies CPU and Memory requests and limits, and requests strictly equal limits. Safest against eviction.
2. **Burstable (Medium Priority):** At least one container specifies requests or limits, and requests are less than limits.
3. **BestEffort (Lowest Priority):** No container specifies any requests or limits. These pods are killed first when node memory pressure occurs.

CHAPTER 32–35: ADVANCED SCHEDULING MECHANISMS

1. NodeSelector vs. NodeAffinity

- `nodeSelector` : Simple key-value label equality (`disk: ssd`).
- `nodeAffinity` : Expressive rule syntax using operators (`In` , `NotIn` , `Exists` , `DoesNotExist` , `Gt` , `Lt`):
    - `requiredDuringSchedulingIgnoredDuringExecution` : Hard filtering predicate.
    - `preferredDuringSchedulingIgnoredDuringExecution` : Soft scoring priority.

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: topology.kubernetes.io/zone
            operator: In
            values: ["us-east-1a", "us-east-1b"]
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 80
        preference:
          matchExpressions:
          - key: instance-type
            operator: In
            values: ["compute-optimized"]
```

2. Taints and Tolerations

- **Taints** repel pods from nodes:

  ```
  kubectl taint nodes worker-01 dedicated=special:NoSchedule
  ```

  Taint Effects:

  - `NoSchedule`: New pods without matching toleration are not placed on the node. Existing pods remain.
  - `PreferNoSchedule`: Soft preference; scheduler tries to avoid placing pods without toleration.
  - `NoExecute`: Pods without matching toleration are immediately evicted from the node.

- **Tolerations** allow pods to be scheduled on tainted nodes:

  ```
  spec:
    tolerations:
    - key: "dedicated"
      operator: "Equal"
      value: "special"
      effect: "NoSchedule"
  ```

3. Pod Affinity and Anti-Affinity

Co-locates or isolates pods relative to other pods within a topological domain:

```
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: app
            operator: In
            values: ["web-frontend"]
        topologyKey: "kubernetes.io/hostname"
```

Effect: Guarantees that two `web-frontend` replicas will never be scheduled on the same physical host.

CHAPTER 36: PRIORITYCLASSES AND PREEMPTION

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority-apps
value: 1000000
globalDefault: false
description: "Mission-critical applications that pre-empt batch workloads."
```

When node capacity is exhausted, the scheduler identifies lower-priority pods, issues eviction requests, and schedules the `high-priority-apps` pod onto the reclaimed capacity.

CHAPTER 37: HORIZONTAL POD AUTOSCALER (HPA)

Autoscales workload replicas based on observed CPU/Memory utilization reported by `metrics-server`:

```
# Imperative HPA creation
kubectl autoscale deployment order-service --cpu-percent=75 --min=2 --max=10
```

Declarative HPA Manifest (autoscaling/v2):

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 75
```

CHAPTER 38: RESOURCEQUOTAS AND LIMITRANGES

1. LimitRange (Container-Level Bounds)

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: mem-limit-range
  namespace: development
spec:
  limits:
  - default:
      memory: 512Mi
      cpu: 500m
    defaultRequest:
      memory: 256Mi
      cpu: 250m
    max:
      memory: 1Gi
      cpu: 1000m
    min:
      memory: 128Mi
      cpu: 100m
    type: Container
```

2. ResourceQuota (Namespace Aggregate Limit)

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: development
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "10"
```

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART IV: SERVICES AND NETWORKING

CHAPTERS 39–45: NETWORKING FUNDAMENTALS, CNI, SERVICES, COREDNS & ENDPOINTSLICES

CHAPTER 39–41: KUBERNETES NETWORKING & CNI ARCHITECTURE

1. Container Network Interface (CNI)

CNI is a standard specification defining plugins to configure network interfaces when containers are started and destroyed.

- Located at: `/opt/cni/bin/` (Executables: `bridge`, `loopback`, `host-local`, `calico`, `flannel`).
- Network configurations located at: `/etc/cni/net.d/` (Evaluated alphabetically in lexicographical order).

2. Pod-to-Pod Communication Across Nodes

1. Pod A (`10.244.1.5`) sends a packet to Pod B (`10.244.2.10`).
2. The packet traverses `veth0` into the host Linux bridge (`cbr0`).
3. The host routing table directs packets destined for `10.244.2.0/24` to the CNI overlay device (e.g., `flannel.1` or `calico-tun`).
4. The overlay interface encapsulates the inner IP packet inside an outer UDP packet (Port 4789/8472 VXLAN) addressed to Worker Node 2 (`192.168.1.20`).
5. Worker Node 2 decapsulates the UDP packet and delivers the original IP packet to Pod B via its local bridge.

CHAPTER 42 & 43: KUBERNETES SERVICE TYPES & PACKET NAT

1. Service Types Compared

- **ClusterIP (Default):** Allocates a virtual cluster-internal IP reachable only from within the cluster.
- **NodePort:** Allocates a high-range static port (30000–32767) on every node in the cluster. Traffic sent to `<NodeIP>:<NodePort>` is forwarded to the Service.

- **LoadBalancer:** Automatically provisions a cloud load balancer (e.g., AWS NLB, GCP Network Load Balancer) that routes to the NodePort.
- **ExternalName:** Maps the Service to a DNS CNAME record (e.g., `my-db.external-service.com`) without proxying.

2. Service YAML Definition

```
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: production
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
  - name: http
    port: 80         # Port exposed on the ClusterIP
    targetPort: 8080 # Port listening on the container
```

CHAPTER 44 & 45: COREDNS DISCOVERY & ENDPOINTSLICES

1. DNS Resolution Architecture

CoreDNS operates as a redundant, scalable deployment in `kube-system`. Its service IP is typically `10.96.0.10`.

- Standard FQDN Structure: `<service-name>.<namespace>.svc.<cluster-domain>`
- Example: `backend-service.production.svc.cluster.local`
- Headless Service Pod FQDN: `<pod-name>.<headless-svc>.<namespace>.svc.<cluster-domain>`

2. Customizing CoreDNS via ConfigMap

To add custom DNS upstreams or stub zones, edit `ConfigMap/coredns`:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: coredns
  namespace: kube-system
data:
  Corefile: |
    .:53 {
        errors
        health {
           lameduck 5s
        }
        ready
        kubernetes cluster.local in-addr.arpa ip6.arpa {
           pods insecure
           fallthrough in-addr.arpa ip6.arpa
           ttl 30
        }
        forward . 8.8.8.8 1.1.1.1
        cache 30
        loop
        reload
        loadbalance
    }
```

3. EndpointSlices vs. Endpoints

Legacy `Endpoints` stored all IP addresses in a single object. Modern Kubernetes utilizes `EndpointSlices` (`discovery.k8s.io/v1`), partitioning endpoint lists into scalable subsets of up to 100 endpoints per slice.

```
# Query EndpointSlices for a specific service
kubectl get endpointslices -l kubernetes.io/service-name=backend-service
```

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART IV: SERVICES AND NETWORKING

CHAPTERS 46–49: INGRESS, GATEWAY API, NETWORKPOLICIES & NETWORK TROUBLESHOOTING

CHAPTER 46: INGRESS AND INGRESS CONTROLLERS

1. Ingress Resource Architecture

An Ingress resource (`networking.k8s.io/v1`) defines routing rules from outside the cluster to internal Services. An **Ingress Controller** (e.g., ingress-nginx) must be installed to evaluate and enforce these rules.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: corporate-ingress
  namespace: web
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: store.example.com
    http:
      paths:
      - path: /products
        pathType: Prefix
        backend:
          service:
            name: product-service
            port:
              number: 80
      - path: /cart
        pathType: Prefix
        backend:
          service:
            name: cart-service
            port:
              number: 80
```

CHAPTER 47: GATEWAY API DEEP DIVE

1. Why Gateway API Replaces Ingress

Ingress suffered from monolithic configuration where cluster administrators and application teams collided in a single resource. Gateway API decouples routing into distinct roles:

1. **Infrastructure Provider:** Defines `GatewayClass`.
2. **Cluster Operator:** Deploys `Gateway` (declares IP addresses, ports, and TLS certs).
3. **Application Developer:** Attaches `HTTPRoute` or `GRPCRoute` (defines paths, weights, and header rewrites).

2. Complete Gateway API Architecture & Manifests

Step 1: Gateway Resource (Cluster Operator)

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: prod-gateway
  namespace: gateway-infra
spec:
  gatewayClassName: standard-gateway
  listeners:
  - name: http-listener
    protocol: HTTP
    port: 80
    allowedRoutes:
      namespaces:
        from: All
```

Step 2: Advanced HTTPRoute with Canary Traffic Splitting (Developer)

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: checkout-route
  namespace: store
spec:
  parentRefs:
  - name: prod-gateway
    namespace: gateway-infra
  hostnames:
  - "checkout.example.com"
  rules:
  # Rule 1: Header-based routing for beta testers
  - matches:
    - headers:
      - name: X-Beta-Tester
        value: "true"
    backendRefs:
    - name: checkout-v2-canary
      port: 8080
  # Rule 2: 90/10 Canary traffic split for regular users
  - matches:
    - path:
        type: PathPrefix
        value: /checkout
    backendRefs:
    - name: checkout-v1-stable
      port: 8080
      weight: 90
    - name: checkout-v2-canary
      port: 8080
      weight: 10
```

3. Cross-Namespace Authorization via ReferenceGrant

When an `HTTPRoute` references a `Gateway` or `Service` in a different namespace, a `ReferenceGrant` must explicitly allow the reference:

```yaml
apiVersion: gateway.networking.k8s.io/v1beta1
kind: ReferenceGrant
metadata:
  name: allow-store-to-gateway
  namespace: gateway-infra
spec:
  from:
  - group: gateway.networking.k8s.io
    kind: HTTPRoute
    namespace: store
  to:
  - group: gateway.networking.k8s.io
    kind: Gateway
```

CHAPTER 48: NETWORKPOLICIES

1. The Default-Deny Security Model

By default, all pods in Kubernetes can receive and send traffic without restriction. Applying a `NetworkPolicy` isolates matching pods, switching them to default-deny.

Default-Deny All Ingress Traffic:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: secure-zone
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

Fine-Grained 3-Tier NetworkPolicy: Allow ingress to `db` pods on port 5432 *only* from `backend` pods in namespace `api`:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-backend-to-db
  namespace: data
spec:
  podSelector:
    matchLabels:
      role: database
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          environment: production
      podSelector:
        matchLabels:
          app: backend
    ports:
    - protocol: TCP
      port: 5432
```

CRITICAL SYNTAX NOTE: In the `from` list, separating `namespaceSelector` and `podSelector` with a hyphen (`-`) creates an **OR** relationship. Omitting the hyphen between them creates an **AND** relationship!

CHAPTER 49: NETWORK TROUBLESHOOTING RUNBOOK

```
# 1. Test intra-pod TCP connectivity
kubectl run net-probe --rm -it --image=nicolaka/netshoot -- nc -zvw3 10.244.1.20 80

# 2. Inspect host iptables forwarding rules for drops
sudo iptables -L FORWARD -n -v | grep DROP

# 3. Check CNI IP allocation leaks
cat /var/lib/cni/networks/k8s-pod-network/*

# 4. Verify Gateway API route status
kubectl describe httproute checkout-route -n store
```

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART V: STORAGE ARCHITECTURE

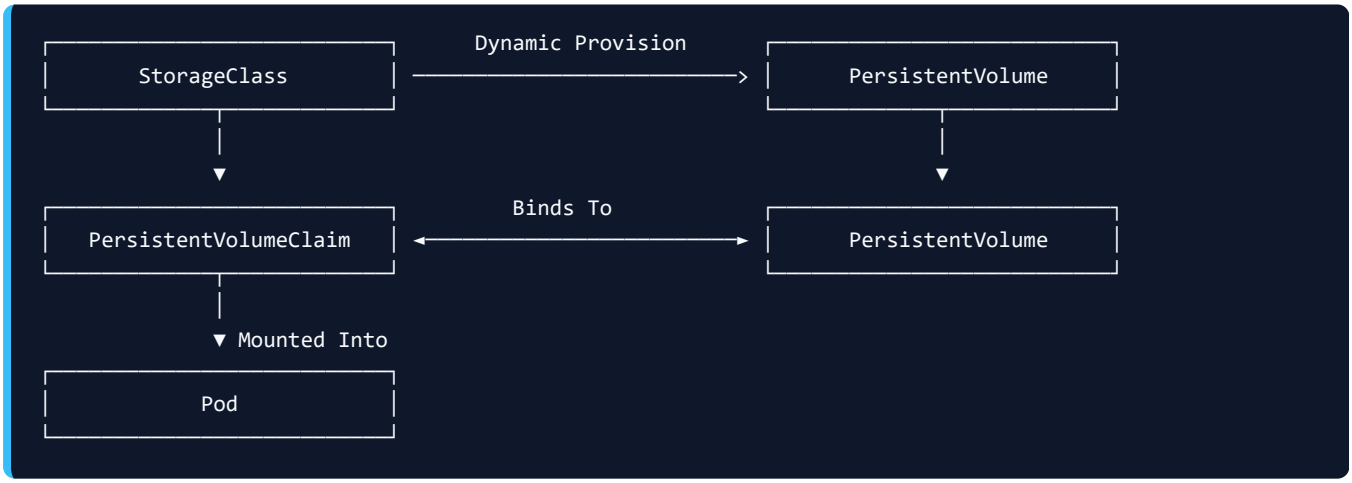
CHAPTERS 50–59: VOLUMES, PV, PVC, STORAGECLASSES, CSI & READWRITEONCEPOD

CHAPTER 50–54: STORAGE ARCHITECTURE, PV, PVC & STORAGECLASSES

1. Learning Objectives

- Understand how storage is decoupled from Pods via PersistentVolumes and PersistentVolumeClaims.
- Contrast static volume provisioning with automated dynamic provisioning.
- Master StorageClass volume binding modes and reclaim policies.

2. The PersistentVolume Lifecycle



3. Storage Access Modes

- `ReadWriteOnce` (RWO): Volume can be mounted as read-write by a single node. Multiple pods on that same node can read and write concurrently.
- `ReadOnlyMany` (ROX): Volume can be mounted read-only by many nodes simultaneously.
- `ReadWriteMany` (RWX): Volume can be mounted read-write by many nodes simultaneously (requires distributed filesystems like NFS, CephFS, or GlusterFS).
- `ReadWriteOncePod` (RWOP): **GA in modern Kubernetes**. Restricts volume access to a *single pod in the entire cluster*. Essential for databases preventing multi-pod write corruption.

4. Reclaim Policies

- `Retain`: When the PVC is deleted, the PV remains intact in `Released` status. Data is preserved for manual recovery.
- `Delete`: When the PVC is deleted, the underlying cloud storage asset is automatically destroyed.

CHAPTER 55: DYNAMIC PROVISIONING & STORAGECLASSES

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-nvme
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
reclaimPolicy: Delete
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: database-pvc
  namespace: database
spec:
  storageClassName: fast-nvme
  accessModes:
  - ReadWriteOncePod
  resources:
    requests:
      storage: 50Gi
```

CRITICAL CKA CONCEPT: `volumeBindingMode: WaitForFirstConsumer` prevents topology mismatches. The PV is not provisioned until a Pod referencing the PVC is scheduled onto a node, ensuring the disk is allocated in the node's exact availability zone.

CHAPTER 58: CONTAINER STORAGE INTERFACE (CSI)

CSI is an open standard allowing storage vendors to develop plugins out-of-tree:

1. **CSI Controller (Control Plane):** Calls storage provider APIs to create and attach disks (`CreateVolume`, `ControllerPublishVolume`).
2. **CSI Node Daemon (Worker Node):** Formats filesystem and bind-mounts device into the container (`NodeStageVolume`, `NodePublishVolume`).

CHAPTER 59: STORAGE TROUBLESHOOTING RUNBOOK

Symptom	Probable Root Cause	Verification & Fix
PVC stuck in `Pending`	StorageClass missing or `WaitForFirstConsumer` active	Check events: `kubectl describe pvc <name>`. If waiting for consumer, schedule a pod using the PVC.
Pod stuck in `ContainerCreating` with `Multi-Attach error`	Previous node crashed without releasing volume lock	Inspect volume attachments: `kubectl get volumeattachments`. Force delete old dead pod.
`VolumeExpansionFailed`	StorageClass has `allowVolumeExpansion: false` or filesystem does not support online resize	Edit StorageClass to enable expansion; restart pod so kubelet expands filesystem.

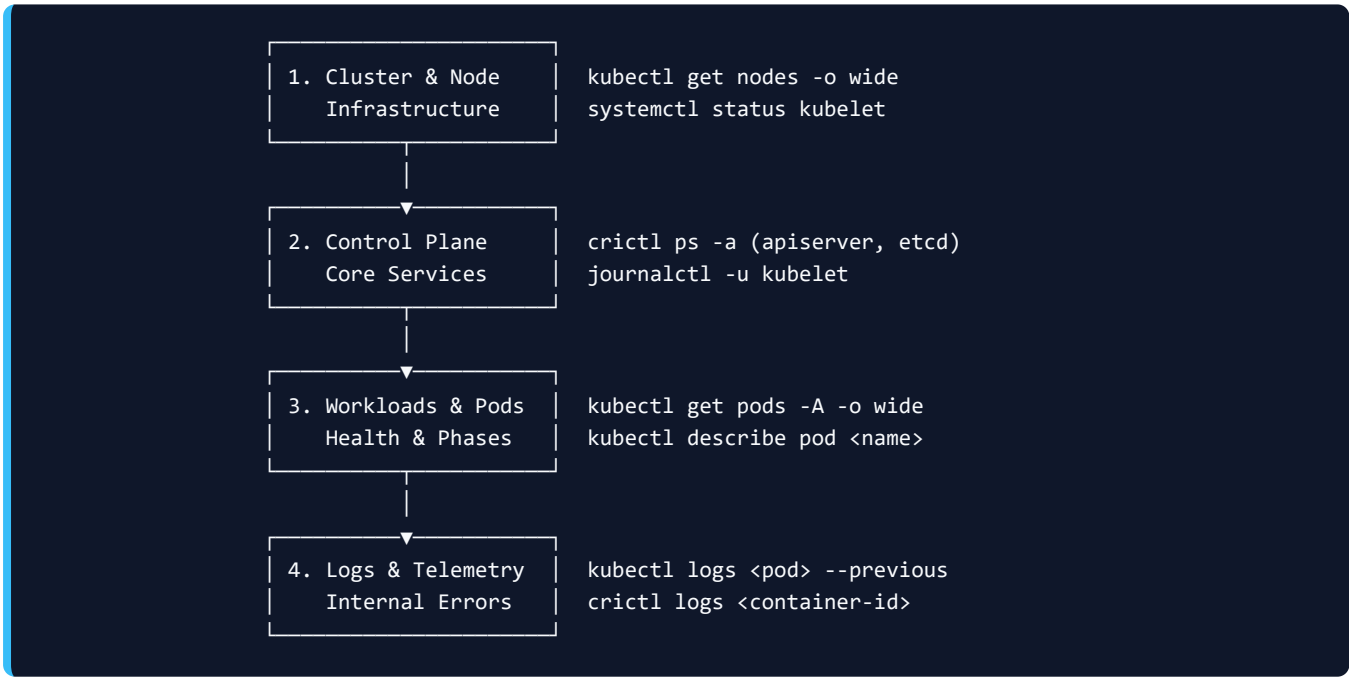
VOLUME I: THE COMPLETE CKA TEXTBOOK

PART VI: COMPREHENSIVE TROUBLESHOOTING

CHAPTERS 60–66: METHODOLOGY, WORKLOADS, NODES, KUBELET & RUNTIME

CHAPTER 60: SYSTEMATIC KUBERNETES TROUBLESHOOTING METHODOLOGY

Troubleshooting accounts for **30% of the CKA exam**. Never guess or execute random commands. Always execute the **Hierarchical Diagnostic Pyramid**:



CHAPTER 61 & 62: POD AND DEPLOYMENT TROUBLESHOOTING

Diagnostic Decision Matrix:

1. `CrashLoopBackOff`:

- Step 1: `kubectl logs <pod-name> --previous`
- Step 2: `kubectl describe pod <pod-name>` (Check Exit Code: 137 = OOMKilled; 0 = Batch process exited; 1 = App error).

2. **`ImagePullBackOff`:**
   - Check image spelling, tag existence, and `imagePullSecrets`.
3. **`CreateContainerConfigError`:**
   - Referenced `ConfigMap` or `Secret` does not exist in the pod's namespace.
4. **Deployment Replicas Not Updating:**
   - Run `kubectl rollout status deployment/<name>`.
   - Check for resource quota violations or failing readiness probes holding back rolling updates.

CHAPTER 63–66: NODE, KUBELET & CONTAINER RUNTIME TROUBLESHOOTING

1. Diagnosing Node `NotReady`

```
# 1. Inspect conditions
kubectl describe node <node-name> | grep -A 6 Conditions

# 2. SSH into node
ssh <node-name>

# 3. Check kubelet service
sudo systemctl status kubelet
sudo journalctl -u kubelet -n 50 --no-pager

# 4. Check containerd runtime
sudo systemctl status containerd
sudo crictl pods
```

2. Kubelet Cgroup Driver Mismatch

If `journalctl -u kubelet` reports `failed to run Kubelet: misconfiguration: kubelet cgroup driver: "cgroupfs" is different from docker/containerd cgroup driver: "systemd"`:

```
# Fix /var/lib/kubelet/config.yaml
sudo sed -i 's/cgroupDriver: cgroupfs/cgroupDriver: systemd/g' /var/lib/kubelet/config.yaml
sudo systemctl restart kubelet
```

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART VI: COMPREHENSIVE TROUBLESHOOTING

CHAPTERS 67–72: API SERVER, ETCD, NETWORKING & INTEGRATED FAILURE DRILLS

CHAPTER 67 & 68: API SERVER & ETCD OUTAGE RECOVERY

1. Recovering a Crashed kube-apiserver

When `kubectl` commands return `The connection to the server 192.168.1.10:6443 was refused`:

1. SSH into the control plane host.
2. The API server runs as a static pod. Inspect `/etc/kubernetes/manifests/kube-apiserver.yaml`.
3. Check runtime container status:

```
sudo crictl ps -a --name kube-apiserver
sudo crictl logs $(sudo crictl ps -a --name kube-apiserver -q | head -n 1)
```

4. Common failure causes:
   - Typo in flag arguments inside `/etc/kubernetes/manifests/kube-apiserver.yaml`.
   - Expired certificates in `/etc/kubernetes/pki/`.
   - `etcd` static container is down or unresponsive on port 2379.

2. Restoring etcd from Corrupted State

Execute the proven disaster recovery workflow:

```
ETCDCTL_API=3 etcdctl snapshot restore /opt/backup.db \
  --data-dir=/var/lib/etcd-recovered \
  --initial-cluster="controlplane=https://127.0.0.1:2380" \
  --initial-cluster-token="k8s-cluster" \
  --initial-advertise-peer-urls="https://127.0.0.1:2380" \
  --name="controlplane"

sudo sed -i 's|path: /var/lib/etcd|path: /var/lib/etcd-recovered|g' /etc/kubernetes/manifests/etcd.yaml
```

CHAPTER 69 & 70: NETWORKING, DNS & STORAGE TROUBLESHOOTING

1. CoreDNS Failure Drill

- **Symptom:** Application pods throw `Name or service not known`.
- **Root Cause 1:** CoreDNS pods stuck in `CrashLoopBackOff` with `Loop ... detected`. Occurs when host `/etc/resolv.conf` loops back to `127.0.0.53`. Update host DNS to external IP (e.g. `8.8.8.8`) and restart CoreDNS.
- **Root Cause 2:** NetworkPolicy in application namespace lacks egress rule on UDP port 53.

2. Service Endpoints Missing

- **Symptom:** `curl http://<service-ip>` hangs or drops connection.
- **Diagnostic:** `kubectl get endpoints <service-name>`. If `<none>`, the service label selector does not match the pod labels or the pods are failing readiness probes.

CHAPTER 71 & 72: INTEGRATED MULTI-COMPONENT FAILURE DRILLS

Master Integrated Drill: The "Blackout" Control Plane

Scenario: A cluster operator accidentally ran a cleanup script on the control plane node. The cluster is completely down:

- `kubectl` returns connection refused.
- `kubelet` is inactive.
- `etcd` data directory was wiped.

Step-by-Step Resolution Runbook:

1. **Restart Kubelet Daemon:**

```
sudo systemctl daemon-reload
sudo systemctl restart kubelet
sudo systemctl status kubelet
```

2. **Verify Static Pod Manifest Directory:** Inspect `/var/lib/kubelet/config.yaml` and confirm:

```
staticPodPath: /etc/kubernetes/manifests
```

3. **Restore etcd Database from Latest Snapshot:**

```
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-snapshot.db \
  --data-dir=/var/lib/etcd-restored \
  --initial-cluster="master=https://127.0.0.1:2380" \
  --name="master"
```

4. **Point etcd Manifest to Restored Data:** Update hostPath in `/etc/kubernetes/manifests/etcd.yaml` to `/var/lib/etcd-restored`.
5. **Verify kube-apiserver Comes Online:** Wait 30 seconds for kubelet to launch static containers. Verify with:

```
sudo crictl ps | grep kube-apiserver
kubectl get nodes
```

6. **Uncordon Nodes & Verify Workload Healing:**

```
kubectl get pods -A
```

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART VII: EXAM PREPARATION & PSI ENVIRONMENT MASTERY

CHAPTERS 73–77: EXAM STRATEGY, TIME MANAGEMENT & PSI REMOTE DESKTOP

CHAPTER 73: UNDERSTANDING THE CKA CERTIFICATION ENVIRONMENT

1. The Official Remote Workstation

The CKA examination is delivered via the PSI Secure Browser, connecting candidates to a remote Linux desktop (Ubuntu 22.04 LTS running XFCE):

- **Terminal:** Standard XFCE Terminal with `bash`.
- **Editor:** `vim` or `nano` (VS Code is present but extensions are disabled; `vim` is recommended for maximum speed).
- **Browser:** Single-tab Chromium browser restricted to official documentation domains:
  - `https://kubernetes.io/docs/`
  - `https://kubernetes.io/blog/`
  - `https://helm.sh/docs/`
- **Clipboard:** Cut and paste operates within the remote desktop using standard Linux shortcuts (`Ctrl+Shift+C` and `Ctrl+Shift+V`).

CHAPTER 74: TIME MANAGEMENT & COMMAND-LINE SPEED

1. The 120-Minute Time Budget

The examination contains 15 to 20 performance tasks:

- Average time per task: **6 to 8 minutes**.
- The **3-Minute Rule**: If you cannot formulate a clear solution path within 3 minutes of reading a task, **FLAG IT AND MOVE ON**.

- High-weight troubleshooting tasks (e.g., control plane outage, cluster upgrade, etcd restore) carry up to 8% to 10% each. Low-weight pod configuration tasks carry 3% to 4%.

2. Essential Time-Saving Aliases

At the start of the exam, run:

```
alias k=kubectl
export do="--dry-run=client -o yaml"
export now="--force --grace-period=0"
```

CHAPTER 75: OFFICIAL DOCUMENTATION NAVIGATION STRATEGY

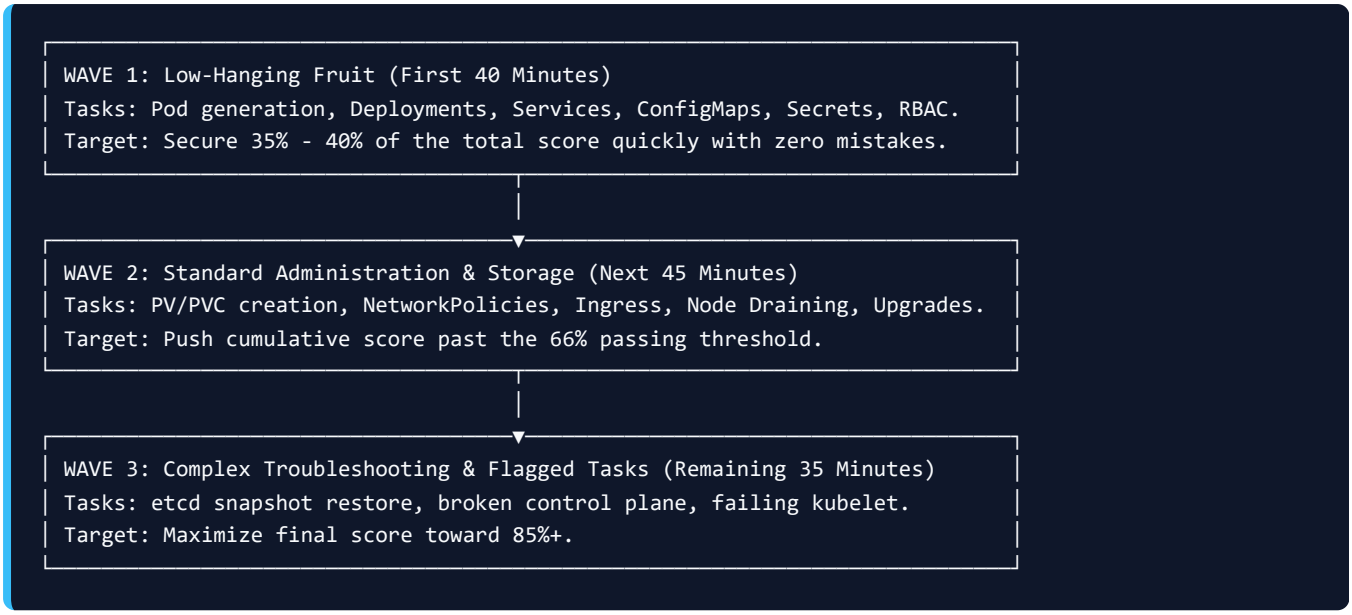
Do not browse the documentation hierarchy during the exam. Use direct keyword search inside the documentation browser tab:

Objective	Documentation Search Keyword	Target Page
NetworkPolicy	"Network Policies"	Concepts -> Services, Networking -> Network Policies
Ingress	"Ingress"	Concepts -> Services, Networking -> Ingress
Gateway API	"Gateway API"	Concepts -> Services, Networking -> Gateway API
PersistentVolume	"Persistent Volumes"	Concepts -> Storage -> Persistent Volumes
Kubeadm Upgrade	"Upgrading kubeadm clusters"	Tasks -> Administer a Cluster -> Upgrading kubeadm clusters
etcd Backup / Restore	"Operating etcd clusters for Kubernetes"	Tasks -> Administer a Cluster -> Operating etcd clusters

Copy YAML code snippets directly from the documentation using the top-right copy icon on code blocks.

CHAPTER 76: EXAM TASK PRIORITIZATION MATRIX

Execute exam questions in three distinct waves:



CHAPTER 77: THE TOP 10 DEADLIEST CANDIDATE MISTAKES

1. **Forgetting to Switch Context:** Every task begins with a context switch command:

   ```
   kubectl config use-context <context-name>
   ```

   Failing to run this executes the solution on the wrong cluster, awarding **zero points**.
2. **Working in the Wrong Namespace:** Always append `-n <target-namespace>` or verify current namespace.
3. **Using Deprecated Flags:** Running `kubectl ... --record` or using obsolete `batch/v1beta1` APIs.
4. **Writing YAML from Scratch:** Always use `kubectl run ... $do` or copy from official docs.
5. **Overwriting the etcd Data Directory:** Restoring etcd directly into `/var/lib/etcd` without specifying a new target directory.
6. **Forgetting to Uncordon Nodes:** Leaving upgraded nodes cordoned.
7. **Indentation Errors in YAML:** Mixing tabs and spaces in `vim`. Configure `~/.vimrc` first!
8. **Ignoring Pod Readiness:** Assuming a pod is working because phase is `Running`, without verifying readiness probes.
9. **Failing to Verify Cluster Status:** Not testing `kubectl get nodes` or `curl` endpoints before moving to the next task.
10. **Panic and Time Freezing:** Spending 25 minutes stuck on a 4% task instead of flagging it.

VOLUME I: THE COMPLETE CKA TEXTBOOK

PART VII: EXAM PREPARATION & PSI ENVIRONMENT MASTERY

CHAPTERS 78–80: READINESS ASSESSMENT, COMPETENCY DRILLS & FINAL CHECKLIST

CHAPTER 78: FINAL READINESS SELF-ASSESSMENT

Before booking your official CKA examination date, evaluate your speed and confidence against this readiness rubric:

Objective Area	Target Completion Time	Required Accuracy
Generate Pod & Expose as ClusterIP	< 60 seconds	100%
Author 3-Tier Ingress / Gateway API HTTPRoute	< 4 minutes	100%
Configure Role & RoleBinding for ServiceAccount	< 2 minutes	100%
Execute Complete etcd Backup & Snapshot Restore	< 7 minutes	100%
Upgrade Control Plane Node via kubeadm	< 8 minutes	100%
Diagnose and Fix `NotReady` Worker Node	< 5 minutes	100%
Build Fine-Grained Default-Deny NetworkPolicy	< 3 minutes	100%

If any task takes longer than the target time, re-execute the corresponding lab in Volume II.

CHAPTER 79: SPEED DRILLS & IMPERATIVE MUSCLE MEMORY

Speed Drill 1: Rapid Workload Exposure

Generate a Deployment named `api-server` with 3 replicas of `nginx:alpine`, exposed internally on port 80 routing to container port 8080:

```
kubectl create deployment api-server --image=nginx:alpine --replicas=3
kubectl expose deployment api-server --name=api-svc --port=80 --target-port=8080
```

Time to execute: Under 20 seconds.

Speed Drill 2: Diagnostic JSONPath Extraction

Extract all pod names and their IP addresses sorted by IP across all namespaces:

```
kubectl get pods -A -o custom-columns=NAME:.metadata.name,IP:.status.podIP --sort-by=.status.podIP
```

Time to execute: Under 30 seconds.

Speed Drill 3: Instant Node Inspection

Identify which node in the cluster is consuming the highest CPU millicores:

```
kubectl top nodes --sort-by=cpu
```

CHAPTER 80: MASTER 24-HOUR PRE-EXAM CHECKLIST

24 Hours Before the Exam:

- [ ] Verify government photo ID name matches your Linux Foundation profile exactly.
- [ ] Ensure desk is completely clear of paper, secondary monitors, pens, and electronic devices.
- [ ] Test webcam, microphone, and internet bandwidth on PSI Secure Browser compatibility check.
- [ ] Review Volume IV Command Field Guide for quick JSONPath and etcdctl syntax recall.

15 Minutes Before the Exam:

- [ ] Log in to Linux Foundation portal and launch PSI Secure Browser.
- [ ] Complete identity verification and 360-degree room scan with remote proctor.
- [ ] Once terminal access is granted, set up bash aliases immediately:

  ```
  alias k=kubectl
  export do="--dry-run=client -o yaml"
  export now="--force --grace-period=0"
  ```

- [ ] Keep calm, read every task's target context carefully, and execute!